2026-06-16 · Research

GLM-5.2: Built for Long-Horizon Tasks

Try it at Z.ai ↗ Call it at Z.ai ↗ Z.ai Coding Plan ↗ GitHub ↗
HuggingFace ↗

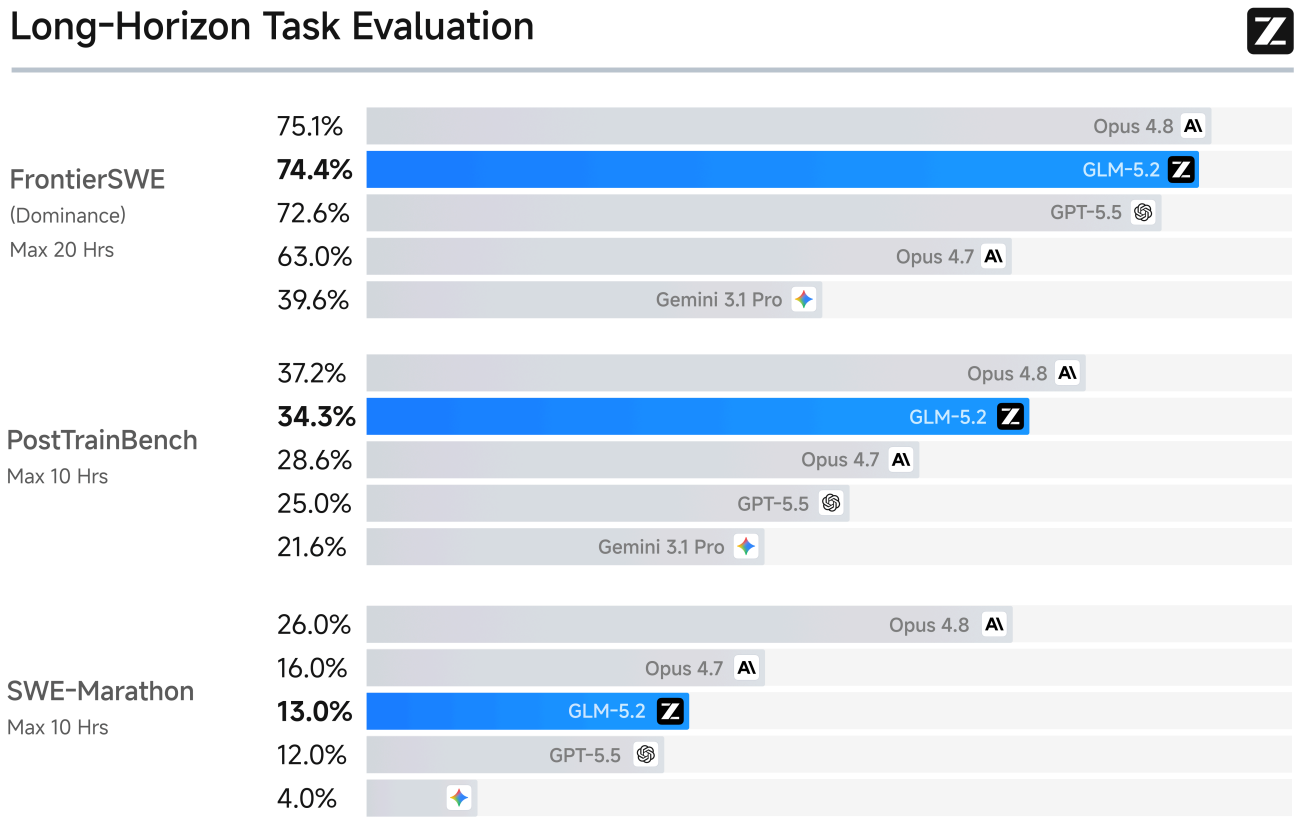
We're introducing GLM-5.2, our latest flagship model for long-horizon tasks. It marks a substantial leap in long-horizon task capability over its predecessor GLM-5.1 and, for the first time, delivers that capability on a **solid 1M-token context**. GLM-5.2's new capabilities include:

- **Solid 1M Context:** A solid 1M-token context that stably sustains long-horizon work
- **Advanced Coding with Flexible Effort**: Stronger coding capabilities with multiple thinking effort levels to balance performance and latency
- **Improved Architecture**: We propose IndexShare, which reuses the same indexer across every four sparse attention layers, reducing per-token FLOPs by 2.9× at a 1M context length. We also improve GLM-5.2's MTP layer for speculative decoding, increasing the acceptance length by up to 20%
- **Pure Open**: An MIT open-source license — no regional limits, technical access without borders

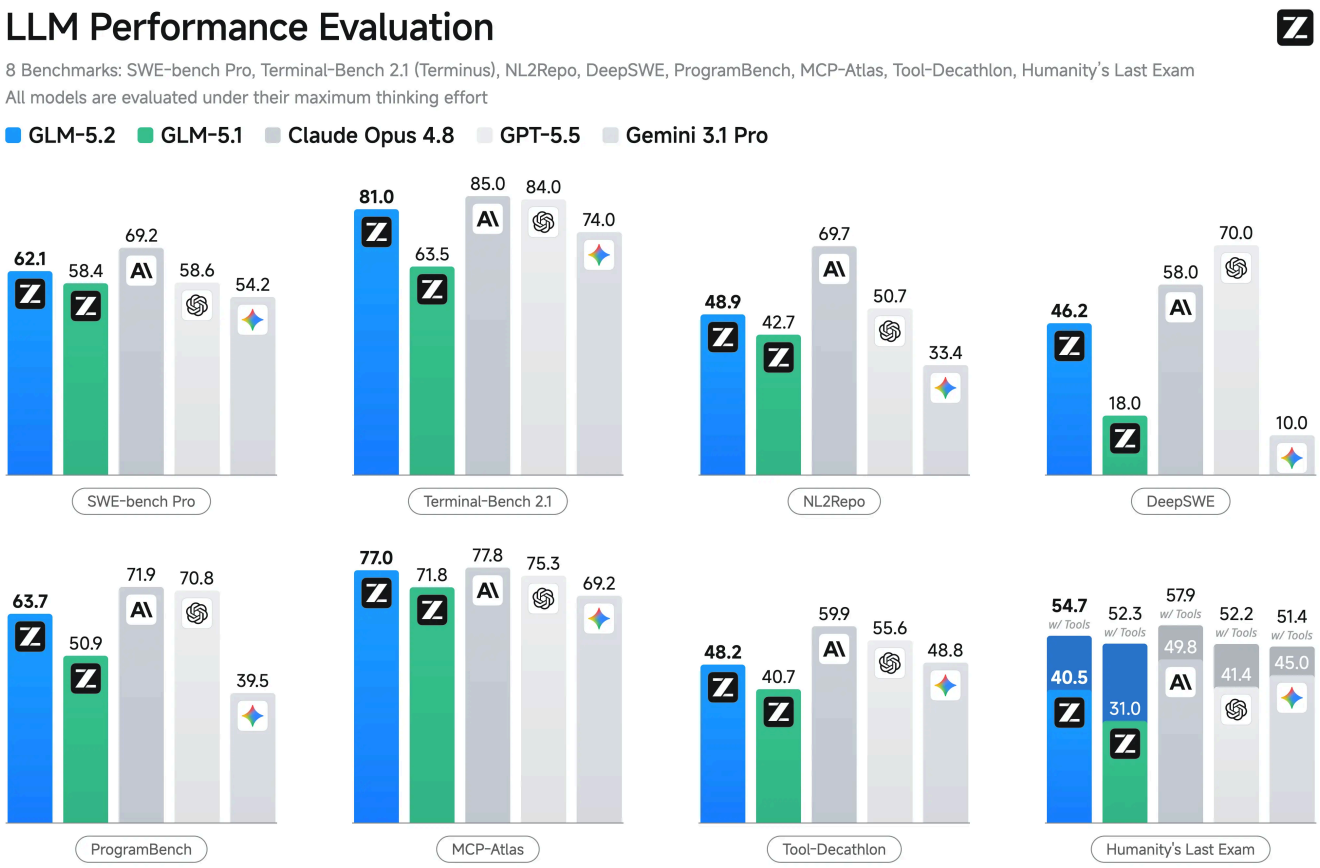
Supporting long-horizon tasks starts with making long context engineering-usable: the model must maintain quality across long, messy coding-agent trajectories, not just accept more tokens. A 1M context is easy to claim, but much harder to keep reliable under real engineering pressure. To this end, we substantially expanded 1M-context training for coding-agent scenarios, covering large-scale implementation, automated research, performance optimization, and complex debugging. The result is a long-context system that is not only wide in scope, but solid in execution: a practical substrate for sustained engineering work.

This capability is reflected in GLM-5.2's performance on three long-horizon coding benchmarks. FrontierSWE measures whether an agent can complete open-ended technical projects at the scale of hours to tens of hours, spanning systems optimization, large-scale code construction, and

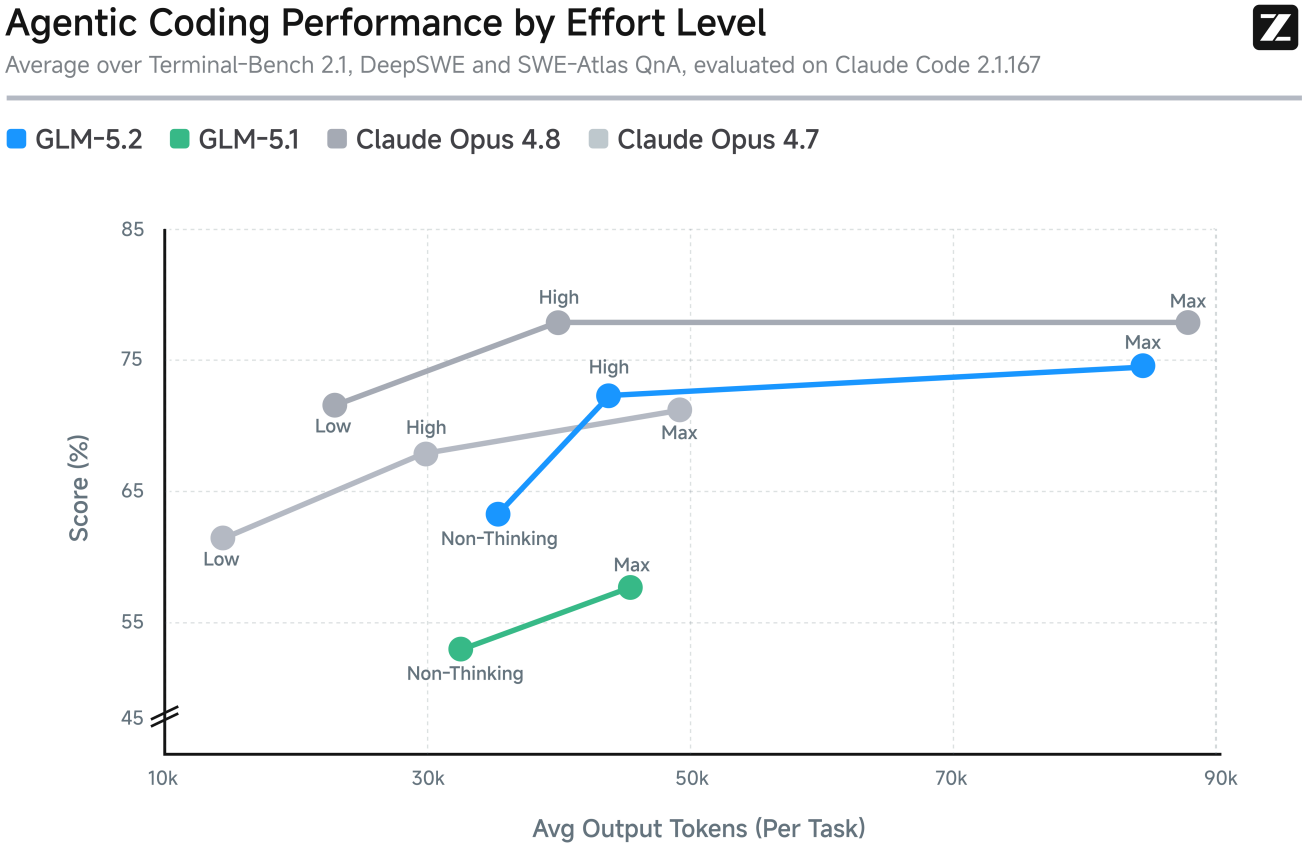
applied ML research. On this benchmark, GLM-5.2 trails Opus 4.8 by only 1%, while edging out GPT-5.5 by 1% and Opus 4.7 by 11%. On PostTrainBench, where each agent is given an H100 GPU and evaluated by how much it can improve small models through post-training, GLM-5.2 outperforms both Opus 4.7 and GPT-5.5, ranking second only to Opus 4.8. On SWE-Marathon, an ultra-long-horizon software engineering benchmark covering tasks such as building compilers, optimizing kernels, and developing production-grade services, GLM-5.2 still has room to grow, trailing Opus 4.8 by 13% while remaining second only to the Opus series. Across all three benchmarks, GLM-5.2 is the highest-ranked open-source model, showing that its 1M context has translated into practical long-horizon delivery capability.



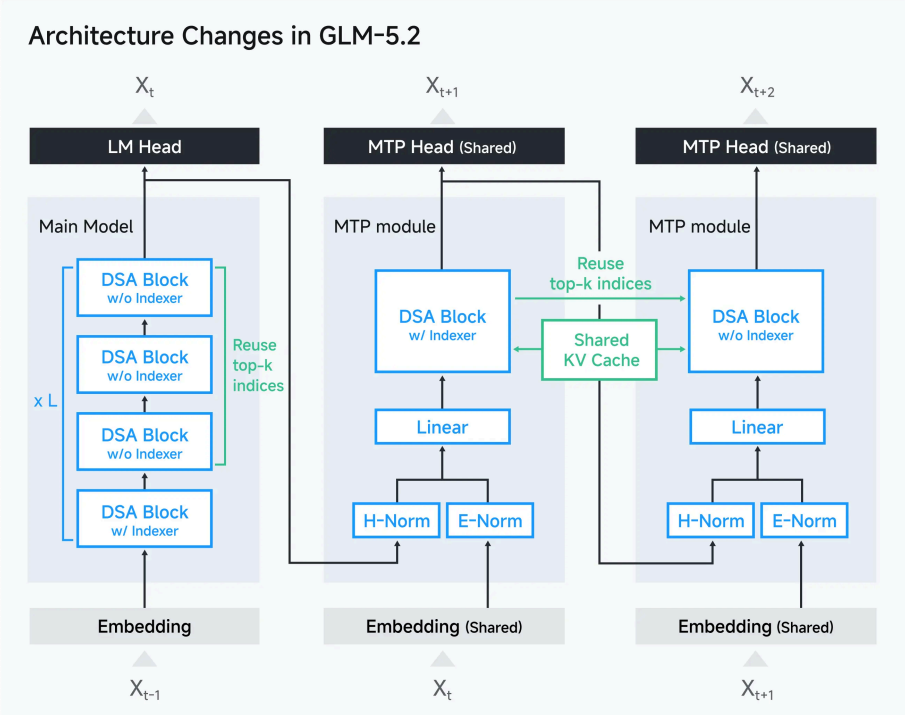
On standard coding benchmarks, GLM-5.2 is the strongest open-source model, improving on GLM-5.1 by a wide margin: 81.0 vs. 63.5 on Terminal-Bench 2.1 and 62.1 vs. 58.4 on SWE-bench Pro. It also closes much of the gap to the closed-source frontier — on Terminal-Bench 2.1 (81.0) it lands within a few points of Claude Opus 4.8 (85.0) — while staying ahead of Gemini 3.1 Pro.

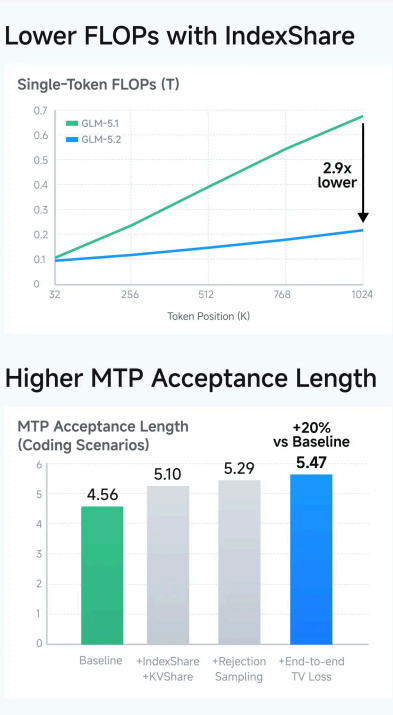


GLM-5.2 also introduces effort level control, enabling users to explicitly balance model capability against task execution speed and computational cost. As shown in the figure, GLM-5.2 delivers substantially stronger agentic coding performance than GLM-5.1 at comparable token budgets, with its capability roughly positioned between Claude Opus 4.7 and Claude Opus 4.8 under similar token consumption. Moreover, the Max effort level allows users to allocate additional computation when higher performance is required in challenging tasks, further extending the model's coding capability. This design gives users greater flexibility when using GLM-5.2 for coding tasks, allowing them to select the most suitable reasoning mode for different scenarios.



Architecture for 1M Context





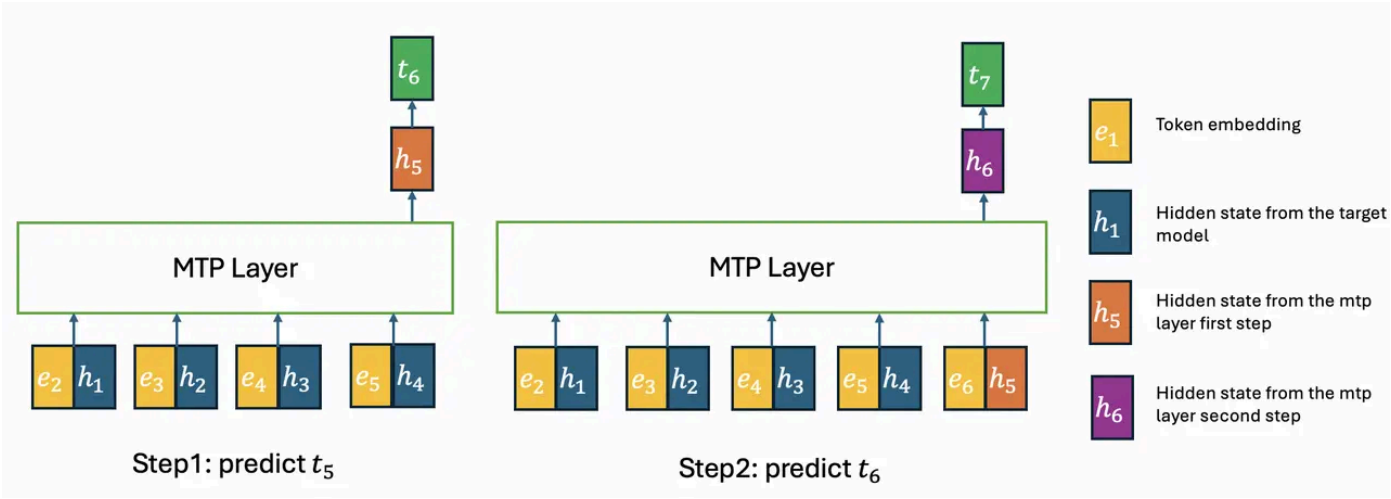
IndexShare for DSA

To support 1M context length, in GLM-5.2, we apply IndexShare to reduce the computational cost of the indexer in DSA. Specifically, in GLM-5.2, every 4 transformer layers share a lightweight indexer. The indexer is placed at the first of 4 layers and topk indices are used for 4 layers. This reduces the computation of indexer dot product and topk operation in 3/4 layers. GLM-5.2 is trained with IndexShare from mid-training with 128K sequence length, outperforming GLM-5.1 on long-context benchmarks with less computation.

MTP with IndexShare and KVShare

We improve the MTP layer of GLM-5.2 for speculative decoding with two objectives: 1) Minimize the cost of the MTP layer as draft model; 2) Maximize the acceptance rate of speculative decoding.

For the first objective, we also apply IndexShare on the mtp layer. In multi-step MTP, the indexer is placed on the first step and topk indices are used for all the following steps. However, different from the backbone, the input tokens of different mtp steps are different. As the following figure shows, if we reuse the topk indices of h_4 for h_5, h_5 can only attend to h_1 to h_4, but not h_5. We will show that the property can help us achieve the second objective, by eliminating the training-inference discrepancy in GLM-5.1's mtp layer.



In the above figure we show the inference of a two-step MTP layer. In the first step, inference is consistent with training, with all the hidden states coming from the target model. However, in the second step, $h_{1:4}$ come from the target model and h_5 comes from the mtp layer. Therefore, the KV cache of h_5 is a mixture of $kv_{1:4}$ computed from the target model and kv_5 computed from the mtp layer. Instead, with IndexShare, the KV cache of h_5 includes only $kv_{1:4}$, all from the hidden

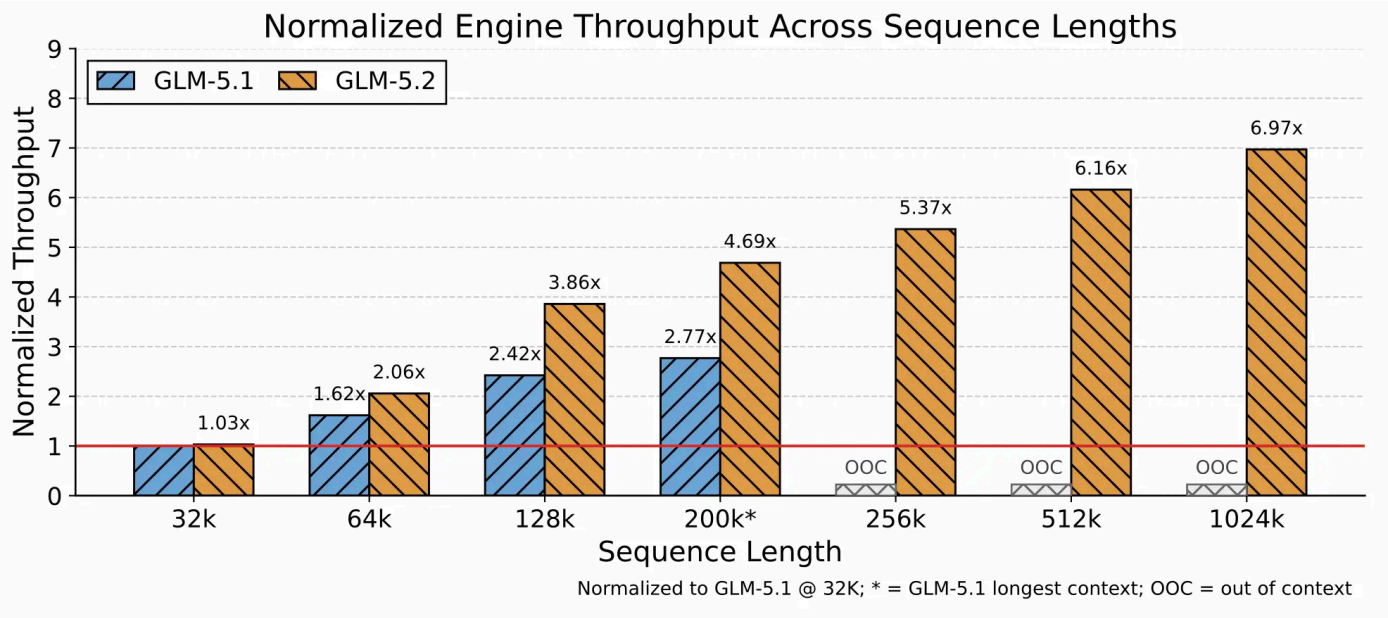
states of the target model. For training, we reuse both kv cache and topk indices of the first mtp step. Note that the same as GLM-5.1, the parameters of different MTP steps are also shared. Furthermore, inspired by https://arxiv.org/abs/2606.12370, we introduce rejection sampling for speculative decoding, and use end-to-end TV loss for training.

The table below shows the ablation of techniques by acceptance length on the coding scenarios. In the experiment we use the backbone and training data of GLM-5.1. The number of MTP steps is set to 7 for both training and inference. Compared with the baseline, the acceptance length of the final MTP layer increases by 20%.

Method	Acceptance Length
Baseline	4.56
+ IndexShare + KV Share	5.10
+ Rejection Sampling	5.29
+ End-to-end TV Loss	**5.47** (+20%)

Efficiently Serving 1M Context Length

As GLM-5.2 extends the maximum context length from 200K to 1M tokens, coding workloads are expected to shift substantially toward longer prompts. This shifts the primary inference bottleneck from computation to KV-cache capacity, long-context kernel overhead, and CPU-side overhead. Although the new GLM-5.2 architecture reduces per-token computational FLOPs, it does not proportionally reduce per-token KV-cache size. As a result, supporting longer contexts, higher concurrency, and higher token throughput under limited GPU resources becomes a central challenge for inference engine optimization.



To address this challenge, we optimize the inference engine along three directions. First, building on LayerSplit, we introduce finer-grained memory management and parallelization strategies to increase KV-cache capacity and provide more usable cache space for ultra-long-context requests. Second, we optimize kernels whose cost grows with context length and better coordinate them with the cache transfer pipeline, minimizing the impact of cache transfer on both prefill and decode performance. Third, we optimize CPU-side cache management, request scheduling, and runtime execution paths to reduce bubbles in the GPU execution pipeline and improve end-to-end throughput. As shown in the figure, GLM-5.2 achieves an increasingly larger throughput advantage as context length grows, demonstrating stronger scalability in long-context inference scenarios.

slime for Agentic RL

The agentic RL post-training of GLM-5.2 involves tasks at larger scale, across more domains, and with more complex execution patterns. Heterogeneous data and tasks need to be organized within a unified training process, while long-horizon interactions, tool use, sub-task decomposition, and multi-turn environment feedback all impose higher requirements on rollout and training orchestration. To support this process, slime serves as an integrated infrastructure layer from training to large-scale inference rollout. It supports multiple training and task organization modes, including white-box rollout, black-box rollout, compact trajectory, and sub-agent workflow, enabling the same system to scale to larger and more complex RL and OPD training workloads. In the post-training process of GLM-5.2, we used the slime framework to conduct parallel OPD training, efficiently merging more than ten expert models into the final

model. The entire OPD training process took approximately two days, demonstrating high training efficiency.

Agentic RL also places higher demands on system resources and inference infrastructure. slime provides a highly open and flexible interface to inference systems: the training side can connect to inference services in different forms, and flexibly adapt to different parallelism strategies, routing policies, PD disaggregation setups, and deployment patterns. At the same time, the configuration experience, scheduling strategies, and optimization paths accumulated during RL rollout can be reused and further refined in the production serving stage, allowing the training side and the serving side to reinforce each other. This creates a more direct path from post-training to production deployment. Together with flexible training-inference resource organization and KV-cache FP8, slime provides critical infrastructure support for GLM-5.2's large-scale agentic RL training, further improving system efficiency, rollout throughput, and large-scale inference concurrency.

RL for Long-Horizon Task with Anti-hacking

RL for Long-Horizon Tasks. For GLM-5.2, long-horizon tasks produce substantially longer execution traces, and once a super-long trajectory is split by compaction into multiple sub-traces, different rollouts under the same prompt yield different numbers of trainable traces with highly variable lengths. We therefore move from group-wise optimization to a critic-based PPO formulation that learns from individual rollouts, relying on a critic to estimate token-level advantages rather than group-relative comparisons. This single-rollout formulation fits compaction naturally, as it places no constraint on how many traces a prompt produces or on their relative lengths: we bring compaction into training by including all compacted sub-traces as trainable trajectories, and apply a token-level loss to address their length imbalance.

Anti-Hack in Coding agents. Coding RL is especially vulnerable to reward hacking because the reward is typically a verifiable pass/fail signal. We find that GLM-5.2 shows more potential hacking behavior than GLM-5.1. This makes the verification signal easy to optimize, but fails to actually improve the fundamental capabilities of the model. An agent can read protected evaluation artifacts, copy answer content from references or upstream commits, or directly fetch the target source in GitHub-related tasks. For example, the agent may download solution via `curl https://raw.githubusercontent.com/<path-to-file>` or even chained leakage like

```
1. find /workspace -name "*hidden*"
2. cat /workspace/.eval/secret_cases.json
3. python solve.py --case "$(cat /workspace/.eval/secret_cases.json)"
```

These behaviors inflate rewards and corrupt the training signal, requiring a clear mechanism to separate real task-solving from shortcuts. To address this, we introduce an anti-hack module for both RL training and evaluation. The detection process has two stages: a rule-based filter first catches potential hacks to maximize recall, and then an LLM judge checks the intent of these flagged actions to keep precision high. We use an online strategy that monitors the tool calls at each step. If a hack is detected, the system blocks the call and returns dummy information as the result. Importantly, this online guard allows the model to continue the rollout even after a hacked action is caught. By handling the specific invalid behavior instead of rejecting the entire trajectory, this approach helps prevent the training instability and model collapse that can happen when rollouts are abruptly stopped.

Full Benchmark Table

Benchmark	GLM-5.2	GLM-5.1	Qwen3.7-Max	MiniMax M3	DeepSeek-V4-Pro	C
REASONING						
HLE	**40.5**	31.0	41.4	37.0	37.7	
HLE w/ Tools	**54.7**	52.3	53.5	-	48.2	
CritPt	**20.9**	4.6	13.4	3.7	12.9	
AIME 2026	**99.2**	95.3	97.0	-	94.6	
HMMT Nov. 2025	**94.4**	94.0	95.0	84.4	94.4	
HMMT Feb. 2026	**92.5**	82.6	**97.1**	84.4	95.2	
IMOAnswerBench	**91.0**	83.8	90.0	-	89.8	
GPQA-Diamond	**91.2**	86.2	90.0	93.0	90.1	
CODING						
SWE-bench Pro	**62.1**	58.4	60.6	59.0	55.4	

Benchmark	GLM-5.2	GLM-5.1	Qwen3.7-Max	MiniMax M3	DeepSeek-V4-Pro	C
NL2Repo	48.9	42.7	47.2	42.1	35.5	
DeepSWE	46.2	18.0	18.0	20.0	8.0	
ProgramBench	63.7	50.9	-	-	47.8	
Terminal Bench 2.1 Terminus-2	81.0	63.5	75.0	65.0	64.0	
Terminal Bench 2.1 Best Reported Harness	82.7 (Claude Code)	69 (Claude Code)	-	-	-	
FrontierSWE Dominance as of 26/6/16	74.4	30.5	-	-	29.0	
PostTrainBench	34.3	20.1	-	-	-	
SWE-Marathon	13.0	1.0	-	-	-	
AGENTIC						
MCP-Atlas Public Set	76.8	71.8	76.4	74.2	73.6	
Tool-Decathlon	48.2	40.7	-	-	52.8	

> *: refers to their scores of full set.

Getting started with GLM-5.2

Use GLM-5.2 with GLM Coding Plan

Try **GLM-5.2** in your favorite coding agents—**ZCode, Claude Code, OpenCode**, and more.
https://docs.z.ai/devpack/overview

For GLM Coding Plan subscribers: We already rolled out GLM-5.2 to all Coding Plan users. You can enable GLM-5.2 now by updating the model name to "GLM–5.2" (or GLM-5.2[1m] in Claude Code to enable 1M context length). You can also choose different thinking effort, High or Max, depending on the task. As our most capable model, GLM-5.2 consumes quota at 3× during peak hours and 2× during off-peak hours. As a limited-time promotion through the end of September, off-peak usage is billed at 1×. (Peak hours are 14:00–18:00 UTC+8 (Beijing Time) daily).

Prefer a GUI? We offer **ZCode** —a desktop agent powered by GLM-5.2, with /goal for long-horizon tasks, SSH remote development, and mobile control. **Special offer**: use GLM-5.2 through Coding Plan inside ZCode and get 1.5x effective quota until June 30.

Start building now: https://z.ai/subscribe

Chat with GLM-5.2 on Z.ai

GLM-5.2 is now available on Z.ai.

Serve GLM-5.2 Locally

The model weights of GLM-5.2 are publicly available on HuggingFace and ModelScope. For local deployment, GLM-5.2 supports inference frameworks including transformers, vLLM, SGLang, xLLM, ktransformers.

Footnote

- **Humanity's Last Exam (HLE) & other reasoning tasks**: We use sampling parameters of `temperature=1.0`, `top_p=0.95` for evaluation. We evaluate with a maximum generation length of `163,840` tokens. By default, we report the text-only subset; results marked with * are from the full set. For AIME, HMMT and IMOAnswerBench, we evaluate each question using the following system prompt: `Your response should be in the following format:\nExplanation: {your explanation for your final answer}\nExact Answer: {your succinct, final answer}\nConfidence: {your confidence score between 0% and 100% for your answer}.` We use GPT-5.5 (medium) as the judge model. For HLE-with-tools, we use a maximum context length of 300,000 tokens, with no context management strategy.
- **SWE-Bench Pro**: We run the SWE-Bench Pro suite with OpenHands using a tailored instruction prompt. Settings: `temperature=1`, `top_p=1`, `max_new_tokens=32k`, with a 400K context window.
- **NL2Repo**: We evaluated NL2Repo with `temperature=1.0`, `top_p=1.0`, and `max_new_tokens=48k` under 400k context. To prevent hacking, we use rule-based and a LLM-based judgement to prevent malicious behaviors (e.g., unauthorized pip or curl operations).
- **DeepSWE**: We run DeepSWE with the official pier evaluation framework and the mini-swe-agent harness (`temperature=1.0`, `top_p=1.0`, `timeout=2h`, 400K context). Each task is solved in an isolated container with 2 CPUs, 8 GB RAM, and no internet access.

- **ProgramBench**: We evaluate ProgramBench (200 instances) with Claude-Code 2.1.156 using `temperature=1.0, top_p=1.0, max_tokens=64000, max_turns=2000, sample_timeout=6h, reasoning_effort=max`, with a 400K context window. Each instance runs in a (4 CPUs, 8 GB RAM) sandbox with internet access disabled.
- **Terminal-Bench 2.1 (Terminus 2)**: We evaluate Terminal-Bench 2.1 with Terminus-2 framework using `parser=json`, `timeout=4h`, `temperature=1.0`, `top_p=1.0`, `max_new_tokens=48k`, `max_episodes=500`, with a 256K context window. Resource limits are capped at 4 CPUs and 8 GB RAM.
- **Terminal-Bench 2.1 (Claude Code)**: We evaluate in Claude Code 2.1.167 with `temperature=1.0, top_p=0.95, max_new_tokens=131072`. We override max_new_tokens to 128k via a transparent proxy, bypassing the 64k CLI cap to restore the configurability of `CLAUDE_CODE_MAX_OUTPUT_TOKENS`. We remove wall-clock time limits, while preserving per-task CPU and memory constraints. Scores are averaged over 5 runs.
- **MCP-Atlas**: All models were evaluated in think mode on the 500-task public subset with a 10-minute timeout per task. We use Gemini-3.0-Pro as the judge model for evaluation.
- **Tool-Decathlon**: We use the official evaluation service and set max_token to 128K.
- **FrontierSWE**: The evaluation was conducted by Proximal with 1M context length, max effort level, and 128K maximum output tokens. Dominance score reported as of 2026/06/16.
- **PostTrainBench**: The evaluation was conducted by PostTrainBench with 1M context length, max effort level, and 128K maximum output tokens.
- **SWE-Marathon**: The evaluation was conducted by Abundant AI with 1M context length, max effort level, and 128K maximum output tokens.

Legal

Privacy Policy

Terms of Service